WARDEN — MCP SANDBOX SERVER

Behind the Scenes

A true story about checking if a security tool really does what it says.

FROM PLAN TO FINISH, EVERY STEP INCLUDED

THE CLAIM	THE PROOF	THE RESULT
A security README	Real commands, run for real	Six real bugs found and fixed

01

EXECUTIVE OVERVIEW

What this project is, and why someone checked it by hand

What warden is

Warden is a small tool. It lets an AI agent run computer commands inside a locked box, called a container. The box is not the real computer. If something goes wrong inside the box, it should not touch anything outside it.

The code for Warden was already written. A file called README.md made a list of promises about how safe the box was. This project was not about writing new code from scratch. It was about checking if those promises were true.

> ◆ *Most tools get built once, then trusted forever. This one got tested before it got trusted.*

Why this mattered

The person who built Warden needed to show it to other people, for a job. They did not want to just say "trust me, it works." They wanted real proof, the kind you can run again yourself and get the same answer.

So instead of reading the code and nodding along, every single claim in the README got tested against a real container, running on a real computer, with real Docker software. If a claim could not be shown working in a real terminal, it did not count as true yet.

02

THE METHOD

How the checking got done, step by step

The rule

Before any code got touched, one rule stayed in place the whole time: **never write down that something works unless you just watched it work.**

STEP	WHAT HAPPENED
Read	Every security file was read start to finish, before anything else
Test	Small scripts were written to run real commands inside the sandbox
Watch	The real output was captured and saved, not just described
Fix	Anything that failed got a real code fix, then got tested again
Prove	Every fix got tested a second time, to make sure it actually worked

The scripts

Three small programs got written just for this checking work. They live in a folder called `scripts/`. Anyone can run them again, right now, on their own computer, and see the same results.

- `verify-timeout-cleanup.js` checks if a stuck command really gets killed.
- `verify-sandbox.js` checks five different safety rules at once.
- `verify-memory-limit.js` checks if the memory limit is a real limit.

03

BUG ONE: THE CONTAINER THAT WOULDN'T DIE

A promise that looked true, but wasn't

The promise

The README said Warden has a hard timeout. If a command runs too long, Warden kills it. The words used were "independent of the process's own behavior." That means: no matter what the command is doing, Warden can always stop it.

The test

A test command got fifty-nine extra seconds to sleep, but Warden only had three seconds to catch it. Sleep for sixty seconds, but let Warden's own timer break in after three.

What actually happened

Warden's timer did go off, right on time. But when the test checked with a real command called `docker ps`, the container was still running, alive two seconds after Warden believed it was gone.

> ◆ *The container was still running. Warden just wasn't watching anymore.*

Why it happened

Warden was only killing its own helper program, the one that starts the container. It was not killing the container itself. Killing the helper is like hanging up the phone on someone but forgetting the oven is still on. The call ends. The oven does not care.

The fix

The fix changed two small things in the file `src/sandbox.js`.

1. Every container now gets its own name when it starts.
2. When the timer goes off, Warden now tells Docker to kill that exact container by name, and waits for that to finish, before hanging up the phone.

Proof it worked

The exact same test ran again after the fix. This time, the moment Warden said the command was done, `docker ps` showed nothing. The container was really gone.

The full real output from both the broken version and the fixed version is saved in `docs/evidence/timeout-before.txt` and `docs/evidence/timeout-after.txt`. Anyone can read the raw proof.

04

BUG TWO: THE MEMORY LIMIT THAT WASN'T

A limit that quietly allowed double

The promise

The README said memory limits stop a runaway program from hurting the computer. Warden lets you set a memory limit, like two hundred fifty six megabytes.

The test

A small program tried to use four hundred megabytes of memory, more than a two hundred fifty six megabyte limit should allow. It should have gotten stopped.

It did not get stopped. It succeeded. Four hundred megabytes, against a two hundred fifty six megabyte limit, and nothing happened.

Why it happened

Docker, the software running the containers, has a second, hidden setting called swap. If you only set a memory limit and never touch the swap setting, Docker quietly doubles your limit on its own. A two hundred fifty six megabyte limit actually behaved like a five hundred twelve megabyte limit.

This is not something Warden's code said anywhere. It is just how Docker behaves by default, and nothing in Warden was turning that default off.

How it got found

To be sure the memory test itself was fair, three separate checks were run together:

CHECK	LIMIT	RESULT	WHY IT MATTERS
Limited run	64 megabytes	Killed	Proves the limit does something
Control run	512 megabytes	Succeeded	Proves the kill was about memory, not a broken command
Real-world check	256 megabytes	Four hundred megabytes got through	Found the actual gap

The fix

The fix added one line to `src/sandbox.js`. It tells Docker to make the swap limit exactly equal to the memory limit, so there is no hidden extra room.

Proof it worked

The same four-hundred-megabyte test ran again after the fix. This time it got killed, exactly as it should have. The full test suite was also rerun to make sure the fix did not break anything that was already working. It did not.

Full raw proof lives in `docs/evidence/memory-limit-oom-inspect.txt`, using a Docker command called `docker inspect`, which can say for certain that a program was killed for using too much memory, and not for some other reason.

05

FIVE MORE CHECKS, AND ONE HONEST SURPRISE

Small things matter too

The checks

Five more promises from the README got tested one at a time, against a real container.

CHECK	WHAT IT PROVES	RESULT
Run a simple command	The sandbox works at all	Passed
Check the user is not root	A break-in can't gain full power	Passed, with a note
Read a system file	Reading files still works	Passed
Write outside the safe folder	Writing should fail everywhere else	Passed, it failed correctly
Try to reach the internet	No network access by default	Passed, it failed correctly

The honest surprise

One command, called `whoami`, is supposed to print your username. Inside the sandbox, it printed an error instead: "unknown uid 1000." At first this looked like a bug.

It was not a bug in Warden. The tiny operating system used inside the sandbox, called Alpine Linux, does not have a name saved for user number one thousand. The safety rule itself, that the program runs as a low powered user and not as the all-powerful root user, was still completely true. It was proven with a different, more reliable command, called `id -u`, which does not need a saved name to work.

> ◆ *A command failing is not always a bug. Sometimes it just means you asked the wrong question.*

Both the failure and the real proof are written down together, in `docs/evidence/functional-checks.txt`. Nothing got hidden or smoothed over.

06

PROVING THE TOOLS ARE REAL, NOT JUST DESCRIBED

Talking to Warden the same way a real AI agent would

The test

Warden is meant to be used by AI tools, through something called MCP. A real MCP testing program talked to Warden exactly the way Claude or another AI assistant would, not through the project's own homemade test scripts.

What got checked

All three of Warden's tools got called for real, one at a time.

TOOL	WHAT IT DOES	RESULT
`run_sandboxed`	Runs a real command in the sandbox	Ran `echo hello`, got `hello` back
`list_audit_log`	Shows a history of what ran	Showed the real command from the step above
`set_egress_allowlist`	Sets which websites a sandbox may reach	Accepted a real list and confirmed it

Every one of these worked exactly as the README says. The full real conversation with the testing program is saved in `docs/evidence/mcp-inspector-tools-list.txt`.

07

SHOWING THE WORK

A real recording, not a mockup

The terminal recording

A tool called `vhs` recorded a real terminal window while the checking scripts actually ran. Nothing in the recording was typed out after the fact or faked. It shows the real six checks passing, and the real audit log printing real entries.

The video

A short video walkthrough builds on that same real recording, with text captions added on top to explain what is happening on screen. No narrator voice was needed, and no invented screens were used. Every frame traces back to a real command that really ran during this session.

Both files live in `docs/demo/`.

08

WHAT IS STILL NOT DONE

Left honest, not hidden

Not every gap gets closed in one sitting, and pretending otherwise would defeat the whole point of this project.

- The list of allowed websites can be saved and viewed, but nothing yet actually blocks or allows real network traffic based on that list. The wiring for that is still missing. This was true before this session and is still true now.
- Warden does not check who is asking it to run a command. It trusts whatever program is talking to it, the same way it always has. Adding a login or identity system is future work, not done here.

The original README already listed both of these honestly, under a section called "Not yet built." Neither one got secretly fixed during this session, so neither one got quietly removed from that list either.

09 A SECOND PASS: PRESSURE TESTING

After the first fixes, the system got attacked on purpose

Why a second pass

Checking that the README's claims were true was the first job. After that job was done, a harder question came next: what happens when someone tries to break this on purpose, in ways the README never even claimed to defend against?

What held up

A loop that tried to spawn five hundred background processes at once, to test the process-count limit, failed immediately with a real kernel error: "can't fork: Resource temporarily unavailable." A tight, endless loop built to burn CPU stayed capped near fifty percent usage, matching the configured limit exactly. Eight sandboxes were run at the same time with no mix-ups.

Four more real bugs

BUG	WHAT WAS WRONG	THE FIX
Unbounded memory	A flooding command could grow the host computer's memory use, with no limit tied to the sandbox's own memory cap	Output is now capped while it streams in, not just after
One bad log line	A single damaged line in the history file crashed the whole log reader	Bad lines are now skipped and flagged, not fatal
Same bug, second file	A damaged settings file crashed the allowlist reader too	A damaged file now safely means "no access," not a crash
Wrong sandbox count	The status command could mistake someone else's unrelated container for a Warden sandbox	Status now checks the sandbox's real name, not just its base image

> ◆ *Every one of these four was found by trying to break something on purpose, not by reading the code and guessing. Every fix was tested again afterward, the same rule as before.*

Full write-up in `docs/learning/pressure-test-audit.md`.

10 FINAL SUMMARY

What was true, what needed fixing, and where to look

What was already true

Most of Warden's promises turned out to be exactly right the first time. No network access by default, a low-powered user inside the sandbox, a read-only file system, and audit logging that happens before a command even runs, all checked out exactly as described.

What needed fixing

Six real problems were found across two rounds of checking, all by actually running the code, not by reading it: two from checking the README's own claims, and four more from trying to break the system on purpose afterward.

Where everything lives

WHAT	WHERE
The original plan for this session	`docs/plan/PLAN.md`
Technical write-ups of every bug	`docs/learning/`
Raw command output, proving every claim	`docs/evidence/`
The real recording and video	`docs/demo/`
The scripts anyone can rerun	`scripts/`
What tools were used to build this	`docs/skills-used/`

> ◆ *Nothing in this document is a summary of a summary. Every claim traces back to a real command, run once, and checked again.*